Bitwise Operators in C

A simple teaching guide: prerequisites, operators, tricks and how to solve scenario-based questions

What is inside	
Part 1	Prerequisites students must know first
Part 2	The 6 bitwise operators with truth tables and examples
Part 3	Bit tricks cheat sheet (check, set, clear, toggle, count)
Part 4	A 5-step method to solve any bitwise question
Part 5	Walkthrough of the 5 scenario questions
Part 6	Common mistakes and practice exercises with answers

Part 1: Prerequisites

Before bitwise operators, students must be comfortable with these four ideas. Spend 15 to 20 minutes revising them.

1.1 Bits and binary numbers

A computer stores every number as a sequence of **bits**. A bit is either **0** or **1**. Each position has a place value that doubles as we move left:

Position	7	6	5	4	3	2	1	0
Place value	128	64	32	16	8	4	2	1
Binary of 25	0	0	0	1	1	0	0	1

25 = 16 + 8 + 1, so 25 in binary is **11001**. Position numbers are counted from the **right**, starting at **0**. This is why 'the k-th bit' always means k places from the right.

1.2 Decimal to binary (repeated division)

Divide by 2 again and again, note the remainders, then read them from bottom to top (this is the question the students already solved).

```
int n = 25;
while (n > 0) {
    printf("%d", n % 2);   // prints remainder (reverse order)
    n = n / 2;
}
```

Bitwise shortcut: `n % 2` is the same as `n & 1`, and `n / 2` is the same as `n >> 1`. Bitwise operators are just a faster, more direct way to work with these bits.

1.3 Binary to decimal

Add the place values where the bit is 1. Example: 1101 = 8 + 4 + 0 + 1 = 13.

1.4 Basics of C needed

- `scanf` and `printf` with %d
- `while` and `for` loops
- `if-else` conditions and the difference between = and ==
- Arithmetic operators: `+ - * / %`

> **CLASS ACTIVITY:** Ask students to convert 6, 13, 25 and 40 to binary on paper before starting Part 2. If they can do this, the rest becomes easy.

Part 2: The Six Bitwise Operators

Bitwise operators work on the **individual bits** of integers, comparing them position by position.

Operator	Name	Short meaning
&	AND	Result bit is 1 only if **both** bits are 1
|	OR	Result bit is 1 if **at least one** bit is 1
^	XOR	Result bit is 1 if the bits are **different**
~	NOT	**Flips** every bit (0 becomes 1, 1 becomes 0)
<<	Left shift	Moves bits left (multiplies by 2 each shift)
>>	Right shift	Moves bits right (divides by 2 each shift)

2.1 AND (&)

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

```
  5 = 0101
  3 = 0011
  -------- &
      0001  = 1
```

Think of it as: a filter or mask. AND keeps a bit only if the mask also has 1 there. **Use it to:** check if a bit is on, check odd/even (`n & 1`).

2.2 OR (|)

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

```
  5 = 0101
  3 = 0011
  -------- |
      0111  = 7
```

Think of it as: turning a switch ON. **Use it to:** set a bit to 1.

2.3 XOR (^)

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

```
  5 = 0101
  3 = 0011
  -------- ^
      0110  = 6
```

Think of it as: a toggle or a 'difference detector'. **Use it to:** flip a bit, swap two numbers, find the unpaired number.

> **REMEMBER:** Two golden rules of XOR: **x ^ x = 0** (same numbers cancel) and **x ^ 0 = x** (zero changes nothing).

2.4 NOT (~)

```
  5 = 00000101
 ~5 = 11111010  = -6   (because of how C stores negative numbers)
```

Rule: `~n = -(n + 1)`. For beginners, just remember this rule. It is used less often in these questions, mainly to clear a bit: `n & ~(1 << k)`.

2.5 Left shift (<<) and Right shift (>>)

```
  5 << 1 = 10      (0101 -> 1010)
  5 << 2 = 20      (0101 -> 10100)
  25 >> 1 = 12     (11001 -> 1100, last bit dropped)
  20 >> 2 = 5      (10100 -> 101)
```

- **n << k** means n x 2^k
- **n >> k** means n / 2^k (integer division)
- **1 << k** creates a number with only the k-th bit ON. This is called a **mask**, and it is the most useful idea in this topic.

Value of k	1 << k	Binary mask
0	1	00001
1	2	00010
2	4	00100
3	8	01000
4	16	10000

2.6 Do not confuse && with &

Logical (&&, ||, !)	Bitwise (&, |, ~)
Works on whole values (true or false)	Works on each bit separately
Result is 0 or 1	Result is a new number
5 && 3 gives 1	5 & 3 gives 1, but 5 & 2 gives 0 while 5 && 2 gives 1

Part 3: Bit Tricks Cheat Sheet

Almost every bitwise question uses one of these tricks. Here **n** is the number and **k** is the bit position counted from the right (starting at 0).

Task	Code	Example (n = 10 = 1010)
Check odd or even	n & 1	10 & 1 = 0, so even
Check if k-th bit is ON	(n >> k) & 1	k=1: (10>>1)&1 = 1, so ON
Set k-th bit (make 1)	n | (1 << k)	k=2: 1010 | 0100 = 1110 = 14
Clear k-th bit (make 0)	n & ~(1 << k)	k=1: 1010 & 1101 = 1000 = 8
Toggle k-th bit (flip)	n ^ (1 << k)	k=1: 1010 ^ 0010 = 1000 = 8
Multiply by 2	n << 1	10 << 1 = 20
Divide by 2	n >> 1	10 >> 1 = 5
Power of two check	(n & (n-1)) == 0	16 & 15 = 0, so YES
Swap without temp	a^=b; b^=a; a^=b;	a=5, b=9 becomes a=9, b=5
Find lone number	XOR all numbers	4^7^4^9^7 = 9

3.1 Why n & (n - 1) works for powers of two

A power of two has exactly one bit ON (like 10000). Subtracting 1 turns that bit OFF and turns all bits to its right ON (01111). So the two numbers share no ON bits, and AND gives 0.

```
16 = 10000        18 = 10010
15 = 01111        17 = 10001
------- &        ------- &
     0 -> YES          10000 (not 0) -> NO
```

3.2 Counting ON bits (set bits) step by step

Check the last bit with n & 1, add to count if it is 1, then shift right to bring the next bit to the end. Repeat until n becomes 0.

n (binary)	n & 1	count	after n >> 1
13 (1101)	1	1	6 (110)
6 (110)	0	1	3 (11)

n (binary)	n & 1	count	after n >> 1
3 (11)	1	2	1 (1)
1 (1)	1	3	0 (stop)

Answer: 13 has **3** bits ON.

3.3 XOR swap step by step (a = 5, b = 9)

Step	Code	a (binary)	b (binary)
Start		0101 (5)	1001 (9)
1	a = a ^ b	1100 (12)	1001 (9)
2	b = a ^ b	1100 (12)	0101 (5)
3	a = a ^ b	1001 (9)	0101 (5)

Part 4: How to Solve Any Bitwise Question (5 Steps)

Teach students to follow the same routine every time. It turns a scary-looking problem into a small, easy one.

Step	What to do	Ask yourself
1. Read	Underline the input, the output and the keywords in the story.	What am I given? What must I print?
2. Convert	Write the sample input in binary on paper.	What do the bits look like?
3. Choose operator	Match the keyword to an operator (table below).	Am I checking, setting, flipping, counting or cancelling?
4. Code	Write scanf, apply the trick, write printf. Add a loop only if the problem needs one.	Is the formula from the cheat sheet?
5. Dry run	Trace the sample input by hand, then test one more case.	Does my output match the sample?

Keyword to operator guide

If the question says...	Use...	Operator
check, is it on, is odd, filter, mask	Extract or test a bit	&
turn on, enable, set, add a flag	Force a bit to 1	|
toggle, flip, switch, different, unique, swap, cancel	Flip or compare bits	^
multiply by 2, double, move position, create mask	Shift bits left	<<
divide by 2, halve, next bit, drop last bit	Shift bits right	>>
turn off, disable, clear	Mask with NOT then AND	& ~

KEY IDEA FOR STUDENTS: Story words (factory, locker, smart home) are only decoration. Remove the story and ask: 'which bit operation is hiding here?'

Part 5: Walkthrough of the 5 Scenario Questions

Q1. Power of Two Memory Block

Keyword: 'power of two'

How to think: Power of two means exactly one bit is ON. Use the trick n & (n-1). If the result is 0, print YES.

Core line: `if ((n & (n - 1)) == 0) printf("YES"); else printf("NO");`

Dry run: 16: 10000 & 01111 = 0 -> YES

Q2. Active Sensors Counter

Keyword: 'count active (1) bits'

How to think: Loop while n > 0. Test the last bit with n & 1, add to count, then shift right with n = n >> 1.

Core line: `while (n > 0) { if (n & 1) count++; n = n >> 1; }`

Dry run: 13 = 1101 -> three 1s -> 3

Q3. Toggle the Light Switch

Keyword: 'toggle k-th bit'

How to think: Build a mask with 1 << k (only the k-th bit ON). XOR with n flips just that bit.

Core line: `n = n ^ (1 << k);`

Dry run: 10 1: 1010 ^ 0010 = 1000 = 8

Q4. Swap Locker Codes

Keyword: 'swap without a third variable'

How to think: Use XOR three times: a = a^b; b = a^b; a = a^b. XOR remembers both values inside one number.

Core line: `a = a ^ b; b = a ^ b; a = a ^ b;`

Dry run: 5 9 -> 12 -> 5 -> 9 -> prints 9 5

Q5. Find the Unpaired Employee ID

Keyword: 'appears twice except one'

How to think: Start result = 0. Read each ID in a loop and XOR it into result. Pairs cancel, leaving the lone ID.

Core line: `for (i = 0; i < n; i++) { scanf("%d", &id); result = result ^ id; }`

Dry run: 4 7 4 9 7 -> 4^4=0, 7^7=0 -> 9

Part 6: Common Mistakes and Practice

6.1 Common mistakes students make

- Using && instead of & (logical instead of bitwise).
- Counting bit positions from the left. Positions always start at **0 from the right**.
- Writing `n == 1 << k` without brackets. Always use brackets: `n ^ (1 << k)`, because == and + bind tighter than bitwise operators.
- Forgetting to initialise `count = 0` or `result = 0` before the loop.
- Using the XOR swap on the same variable twice (like swapping a with a), which makes it 0.
- Thinking ~5 is 10. It is **-6**.
- Forgetting the loop condition `n > 0` when counting bits or converting to binary.

6.2 Practice: find the output by hand (with answers)

No.	Question	Working	Answer
1	12 & 10	1100 & 1010 = 1000	8
2	12 | 10	1100 | 1010 = 1110	14
3	12 ^ 10	1100 ^ 1010 = 0110	6
4	3 << 3	3 x 8	24
5	40 >> 2	40 / 4	10
6	Is 9 odd or even? (use &)	9 & 1 = 1	Odd
7	Toggle bit 0 of 6	0110 ^ 0001 = 0111	7
8	Set bit 3 of 5	0101 | 1000 = 1101	13

6.3 Suggested class plan (90 minutes)

Time	Activity
0 - 15 min	Revise binary conversion (Part 1) with 4 numbers on the board
15 - 40 min	Teach the six operators with truth tables and examples (Part 2)
40 - 55 min	Cheat sheet tricks and the mask idea 1 << k (Part 3)
55 - 75 min	Teach the 5-step method and solve Q1 to Q3 together (Parts 4 and 5)
75 - 90 min	Students solve Q4 and Q5 and the practice table (Part 6)